Artificial Intelligence in Pla...

My Copilot subscription expired on a Tuesday. I didn't renew it.

Medium Search Write

Home
Library
Profile
Stories
Stats

Following
Dev Genius
Python in Plain En...
Generative AI
Realworld AI Use C...
The Useful Tech
Level Up Coding
AI Mind
Deep concept
Top Python Librari...
The Mac Alchemist

Top highlight

Reading list
Implement

More

Media Generation
Python
Business Opportunities
Create new list

BUILD, DON'T SUBSCRIBE.

I Replaced GitHub Copilot W Local Setup. Here's What N Tells You

A private, offline assistant using Gemma 4 + Ollama that changed how I write code.

Kuldeepsinh Jadeja Follow 5 min read · Apr 20, 2026

My GitHub Copilot trial expired on a Tuesday.

I didn't renew it.

Not because I couldn't afford it, but because I'd just spent a weekend running Gemma 4 locally through Ollama...

...and I realized I didn't need to anymore.

No API call.
No latency spikes.
No sending code to the cloud.

Just local.

And the obvious question hit me:
Why am I paying monthly for something I can run on my own machine?

If you're new to local models, I previously tested one in depth here:
"I Ran Gemma 4 Locally. Here's What Nobody's Telling You."

What Is a Local Coding Assistant With Gemma 4 and Ollama?

Whenever I talk about local LLMs, people usually ask:
"Why not just pay the $10?"

Because this isn't just about cost.

A **local coding assistant with Gemma 4 + Ollama is a self-hosted AI coding tool** that runs entirely on your machine.

It lets you:

- **Developers generate Code**
- **Debug Issues**
- **Refactor code**

Without ever sending your code to the cloud making it a private, offline AI coding tool and a real alternative to cloud-based solutions like **GitHub Copilot** and **Claude**.

The 3 Dealbreakers That Made Me Switch

1. **Privacy (Your Code Stays Yours)**

Every cloud AI tool sends your code somewhere.

A **local coding assistant with Gemma 4 and Ollama** keeps:

- **Client code**
- **Internal systems**
- **Proprietary logic**

Completely offline

2. Cost (Kill the Subscription Loop)

Cloud AI tools charge monthly, and costs can skyrocket with heavy use.

A **local coding assistant with Gemma 4 and Ollama** is a one-time setup. You pay for the hardware and software once, and then it's yours to use indefinitely.

- **No subscriptions**
- **No usage limits**
- **One-time setup**

This is the same mindset shift I had when I automated most of my workflow → "**I Automated 80% of My Workflow With AI**"

3. Total Control Over the Engine (Underrated Advantage)

With a **Local LLM for coding**, you control:

- **Model behavior**
- **Latency**
- **Environment**

No rate limits

No downtime

No black box

The benchmark jump from Gemma 3 to Gemma 4 isn't incremental, it's a different tier of capability.

What Makes Gemma 4 Different From Previous Open Models?

Older local models forced trade-offs. Smaller open models were fast but dumb. Smarter ones required expensive hardware and were still inconsistent.

Gemma 4 changes that.

- **Gemma 3 Codeforces ELO: 110**
- **Gemma 4 Codeforces ELO: 2,150**

That's not an improvement

> *That's a capability jump.*

Why It Actually Works

Built-in reasoning

Handles multi-step thinking with high consistency.

The model can generate 4,000+ tokens of internal reasoning.

Native function calling

Makes debugging loops (run → error → fix) consistent.

This removes a problem I talked about before:
AI tools often feel inconsistent: **"Your AI Agent Isn't Dumb. It Has ADHD"**

> Gemma 4 also ships under the Apache 2.0 license, meaning you can use it in commercial projects without worrying about custom terms of service.

Hardware Requirements for a Local Coding Assistant

Let's be direct about this because most guides skip it.

Entry Level

- **8GB RAM**
- **CPU only**

> *Works for basic tasks, but slow.*

Recommended Setup (Best Value)

- **12–16GB VRAM GPU**
- **RTX 3080 / 4070**

> *Ideal for a* ***Local coding assistant with Gemma 4 and Ollama***

High-End

- **80GB GPU (or heavy quantization)**

My Setup:

RTX 4070 Ti + 26B quantized

Result:

- **Responses under ~3 seconds**
- **Smooth debugging + refactoring**

How to Build a Local Coding Assistant (15-Min Setup)

This is the part most people overthink. It's actually four steps. If you have 15 minutes, you can have this running.

Step 1: Install Ollama

For Mac and Linux:

```
curl -fsSL https://ollama.com/install.sh | sh
```

For Windows:

```
irm https://ollama.com/install.ps1 | iex
```

Run this in powershell.

Or refer to the **Ollama website** for GUI installers.

Step 2: Pull the Gemma 4 model

```
ollama pull gemma4:27b
```

Swap `27b` for `9b` or **4b** depending on your hardware. The first pull takes a few minutes. After that, it's cached locally.

Step 3: Start the Ollama server

```
ollama serve
```

This exposes a local API at `http://localhost:11434`. That's it. Your model is now running!

Your offline AI coding tool runs locally.

Step 4: The Sanity Check

Open a new terminal tab and test it:

```
ollama run gemma4:27b "Explain what this code does:
> const fn = arr => arr.reduce((a, b) => a + b, 0)"
```

If it correctly explains the array reduction, your AI is alive and kicking.

Four commands. That's the entire setup. Everything after this is just building.

Connect Your Local Coding Assistant to VS Code (This Changes Everything)

Running prompts in a terminal is cool for testing, but you need this inside your IDE.

Enter **Continue.dev** an incredible open-source Copilot alternative that plugs directly into VS Code and JetBrains.

It integrates directly with any Ollama model.

Install the Continue extension, then point it at your local Ollama server:

```
{
  "models": [
    {
      "title": "Gemma 4 Local",
      "provider": "ollama",
      "model": "gemma4:27b",
      "apiBase": "http://localhost:11434"
    }
  ]
}
```

What Works Well

- **Code explanations**
- **Refactoring**

- **Debugging**
- **Boilerplate**

Where Copilot Still Wins

- **Real-time autocomplete latency**
- **Large repo context/awareness**

My Honest Verdict After 2 Weeks

A **Local coding assistant with Gemma 4 and Ollama** didn't feel like a downgrade.

It felt like **control.**

You:

- **Experiment more**
- **Iterate faster**
- **Share full context freely**

That changes how you work.

Gemma 4 locally isn't a perfect Copilot replacement for every workflow.

> *Some Question which were concerning to me before switching:*

Can a local coding assistant replace Copilot?

Yes. A **local coding assistant with Gemma 4 and Ollama** handles most coding tasks except ultra-fast autocomplete.

Is it really private?

Yes. A **self-hosted AI coding assistant** runs fully offline.

Is a local coding assistant better than cloud AI tools?

For privacy, cost, and control, yes. A **local coding assistant with Gemma 4 and Ollama** gives full ownership and eliminates recurring costs.

No subscription. No cloud. No one else's server. Just your machine, doing the work.

The Real Takeaway

A **local coding assistant with Gemma 4 and Ollama** is no longer experimental.

It's practical.

It's reliable.

And for many developers, it's enough to replace cloud tools.

I opened my bank statement last week.

Noticed the Cloud charge was gone.

I didn't add it back.

Because a **local coding assistant with Gemma 4 and Ollama** already replaced it.

Artificial Intelligence | Programming | Software Development | Ollama | Open Source